

SRG091X&SRG093X 系 列

Linux 蜂窝模块接入用户指导

短距离产品

版本：1.0.0

日期：2026-09-11

状态：临时文件



上海移远通信技术股份有限公司（以下简称“移远通信”）始终以为客户提供最及时、最全面的服务为宗旨。如需任何帮助，请随时联系我司上海总部，联系方式如下：

上海移远通信技术股份有限公司

上海市松江区泗泾镇外婆泾路 8 号 邮编：201601

电话：+86 21 5108 6236

邮箱：info@quectel.com

或联系我司当地办事处，详情请登录：<https://www.quectel.com.cn/contact>。

如需技术支持或反馈我司技术文档中的问题，请随时登录网址：

<https://www.quectel.com.cn/contact?tab=t> 或发送邮件至：support@quectel.com。

前言

移远通信提供该文档内容以支持您的产品设计。您须按照文档中提供的规范、参数来设计产品。同时，您理解并同意，移远通信提供的参考设计仅作为示例。您同意在设计您目标产品时使用您独立的分析、评估和判断。在使用本文档所指导的任何硬软件或服务之前，请仔细阅读本声明。您在此承认并同意，尽管移远通信采取了商业范围内的合理努力来提供尽可能好的体验，但本文档和其所涉及服务是在“可用”基础上提供给您的。您知悉并同意，移远通信可在未事先通知的情况下，自行决定随时增加、修改或重述本文档，增加、修改或重述后的文档对您具有约束力。

使用和披露限制

许可协议

除非移远通信特别授权，否则我司所提供硬软件、材料和文档的接收方须对接收的内容保密，不得将其用于除本项目的实施与开展以外的任何其他目的。

版权声明

移远通信产品和本协议项下的第三方产品可能包含受移远通信或第三方材料、硬软件和文档版权保护的相关资料。除非事先得到书面同意，否则您不得获取、使用、向第三方披露我司所提供的文档和信息，或对此类受版权保护的资料进行复制、转载、抄袭、出版、展示、翻译、分发、合并、修改，或创造其衍生作品。移远通信或第三方对受版权保护的资料拥有专有权，不授予或转让任何专利、版权、商标或服务商标权的许可。为避免歧义，任何形式的购买都不可被视为授予除正常的非独家、免版税的产品使用许可之外的任何许可。对于任何违反保密义务、未经授权使用或以其他非法形式恶意使用所述文档和信息的违法、侵权行为，移远通信有权追究法律责任。

商标

除另行规定，本文档中的任何内容均不授予在广告、宣传或其他方面使用移远通信或第三方的任何商标、商号及名称，或其缩略语，或其仿冒品的权利。

第三方权利

您理解本文档可能涉及一个或多个属于第三方的硬软件和文档（“第三方材料”）。您对此类第三方材料的使用应受本文档的所有限制和义务约束。

移远通信针对第三方材料不做任何明示或暗示的保证或陈述，包括但不限于任何暗示或法定的适销性或特定用途的适用性、平静受益权、系统集成、信息准确性以及与许可技术或被许可人使用许可技术相关的不侵犯任何第三方知识产权的保证。本协议中的任何内容都不构成移远通信对任何移远通信产品或任何其他软硬件、设备、工具、信息或产品的开发、增强、修改、分销、营销、销售、提供销售或以其他方式维持生产的陈述或保证。此外，移远通信免除因交易过程、使用或贸易而产生的任何和所有保证。

隐私声明

为实现移远通信产品功能，特定设备数据将会上传至移远通信或第三方服务器（包括运营商、芯片供应商或您指定的服务器）。移远通信严格遵守相关法律法规，仅为实现产品功能之目的或在适用法律允许的情况下保留、使用、披露或以其他方式处理相关数据。当您与第三方进行数据交互前，请自行了解其隐私保护和数据安全政策。

免责声明

- 1) 移远通信不承担任何因未能遵守有关操作或设计规范而造成损害的责任。
- 2) 移远通信不承担因本文档中的任何因不准确、遗漏、或使用本文档中的信息而产生的任何责任。
- 3) 移远通信尽力确保开发中功能的完整性、准确性、及时性，但不排除上述功能错误或遗漏的可能。除非另有协议规定，否则移远通信对开发中功能的使用不做任何明示、暗示或法定的保证。在适用法律允许的最大范围内，移远通信不对任何因使用开发中功能而遭受的损害承担责任，无论此类损害是否可以预见。
- 4) 移远通信对第三方网站及第三方资源的信息、内容、广告、商业报价、产品、服务和材料的可访问性、安全性、准确性、可用性、合法性和完整性不承担任何法律责任。

版权所有 ©上海移远通信技术股份有限公司 2026，保留一切权利。

Copyright © Quectel Wireless Solutions Co., Ltd. 2026.

文档历史

修订记录

版本	日期	作者	变更表述
-	2026-09-11	Elijah Zhou/ Selene Lei	文档创建
1.0.0	2026-09-11	Elijah Zhou/ Selene Lei	临时版本

目录

文档历史	3
目录	4
表格索引	5
图片索引	6
1 引言	7
2 环境准备	8
2.1. 硬件环境	8
2.2. 硬件改装说明	12
2.3. 软件环境	13
3 移植蜂窝模块驱动	14
3.1. 移植 USB 转串口驱动	14
3.2. 移植 USB 网络驱动	17
3.2.1. 集成 QMI_WWAN 驱动	17
3.2.2. 集成 ECM 驱动	17
4 移植 QConnectManager	19
5 功能验证	21
5.1. 验证上网功能	21
5.2. 验证语音通话功能	28
5.2.1. 硬件条件说明	28
5.2.2. 使能 SAI1 接口和 sound-bt-sco 声卡	30
5.2.3. 配置蜂窝模块 PCM 接口	31
5.2.4. 验证语音通话	31
6 附录 参考文档及术语缩写	33

表格索引

表 1：硬件环境..... 8

表 2：软件环境..... 13

表 3：串口信息..... 28

表 4：参考文档..... 33

表 5：术语缩写..... 33

图片索引

图 1: 安装 SRG091X-TE-A 到 SR-IMXM-EVB.....	9
图 2: 安装 EG25-GL Mini PCIe 模块和 SIM 卡到 SR-IMXM-EVB.....	10
图 3: 安装 EG800AK-EU 模块和 SIM 卡到 UMTS<E-EVB.....	11
图 4: SR-IMXM-EVB 电源接口和 USB-AP 接口.....	12
图 5: SR-IMXM-EVB 硬件改装说明.....	13
图 6: 串口信息.....	27
图 7: 对 SRG091X-TE-A 进行硬件改动以支持 Codec.....	29
图 8: 通过拨码开关切换 PCM 通路.....	29
图 9: 将耳机插入 SR-IMXM-EVB 的耳机接口.....	32

1 引言

移远通信 SRG091X 和 SRG093X 系列模块搭载丰富外设接口，可外接 USB 接口蜂窝模块，实现语音通话与蜂窝数据上网功能。为便于产品功能评估与应用开发，移远通信配套推出对应型号开发板（SRG091X-TE-A 和 SRG093X-TE-A）及 SR-IMXM-EVB。本文档以 SR-IMXM-EVB 为硬件载体，分别搭载不同型号的开发板及蜂窝模块作为示例，介绍如何在 SRG091X 和 SRG093X 系列模块上适配蜂窝模块并进行功能验证。

备注

1. 不同模块支持的 USB 网络驱动和验证方式不同，详情请参考**第 3 章**和**第 5 章**。本文分别以 SR-IMXM-EVB 搭载 SRG091X-TE-A 和 EG21-GL Mini PCIe/EG25-GL Mini PCIe/EG916Q-GL Mini PCIe 模块、以 SR-IMXM-EVB 搭载 SRG093X-TE-A 和 EG800AK-EU 模块为例，介绍在 SRG091X 和 SRG093X 系列模块上适配蜂窝模块的步骤和验证蜂窝模块功能的方法。
2. EG800AK-EU 模块没有 Mini PCIe 接口的开发板，需额外搭配蜂窝模块开发板使用（本文以 UMTS<E-EVB 为例）。蜂窝模块开发板型号仅供参考，若使用其他型号的蜂窝模块开发板，请向对应厂商咨询详细信息。

2 环境准备

2.1. 硬件环境

表 1：硬件环境

硬件名称	数量
移远通信 SRG091X-TE-A	1
移远通信 SRG093X-TE-A	1
移远通信 SR-IMXM-EVB	1
移远通信 UMTS<E-EVB	1
移远通信 EG25-GL Mini PCIe/EG21-GL Mini PCIe/EG916Q-GL Mini PCIe/EG800AK-EU 模块	1
已激活的 SIM 卡	1
蜂窝天线	1
耳机	1
Type-C 转 Type-A 数据线	2
Micro USB 转 Type-C 数据线	1
电源线	1

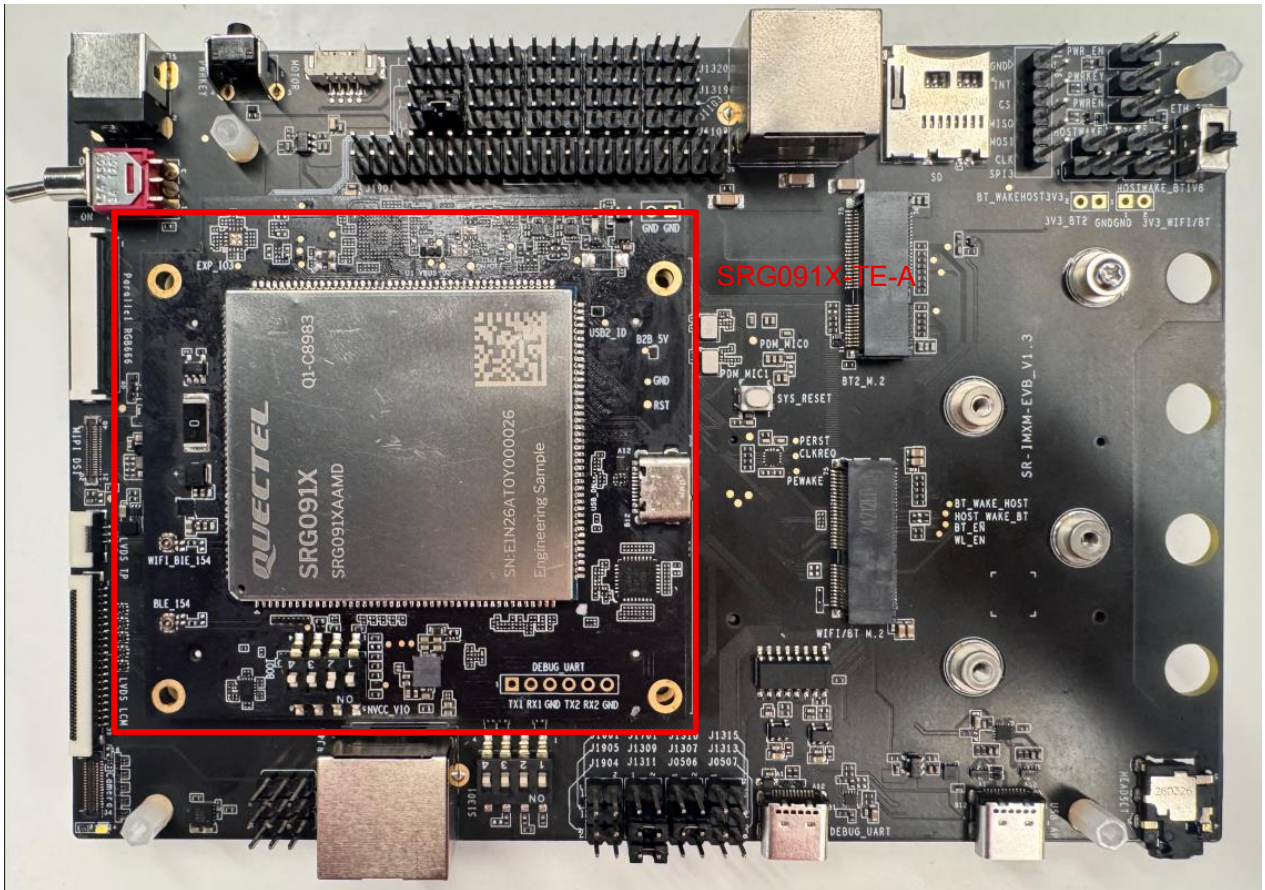


图 1: 安装 SRG091X-TE-A 到 SR-IMXM-EVB

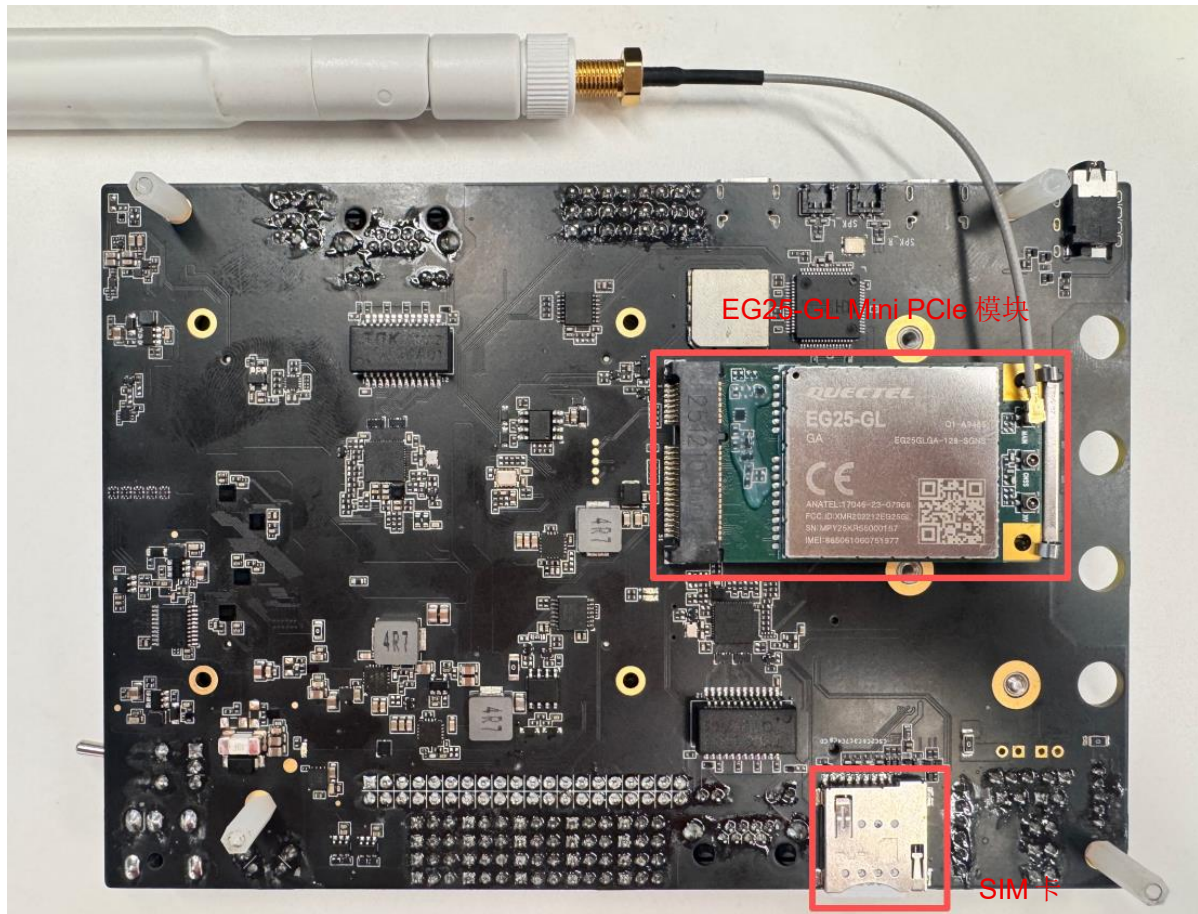


图 2: 安装 EG25-GL Mini PCIe 模块和 SIM 卡到 SR-IMXM-EVB

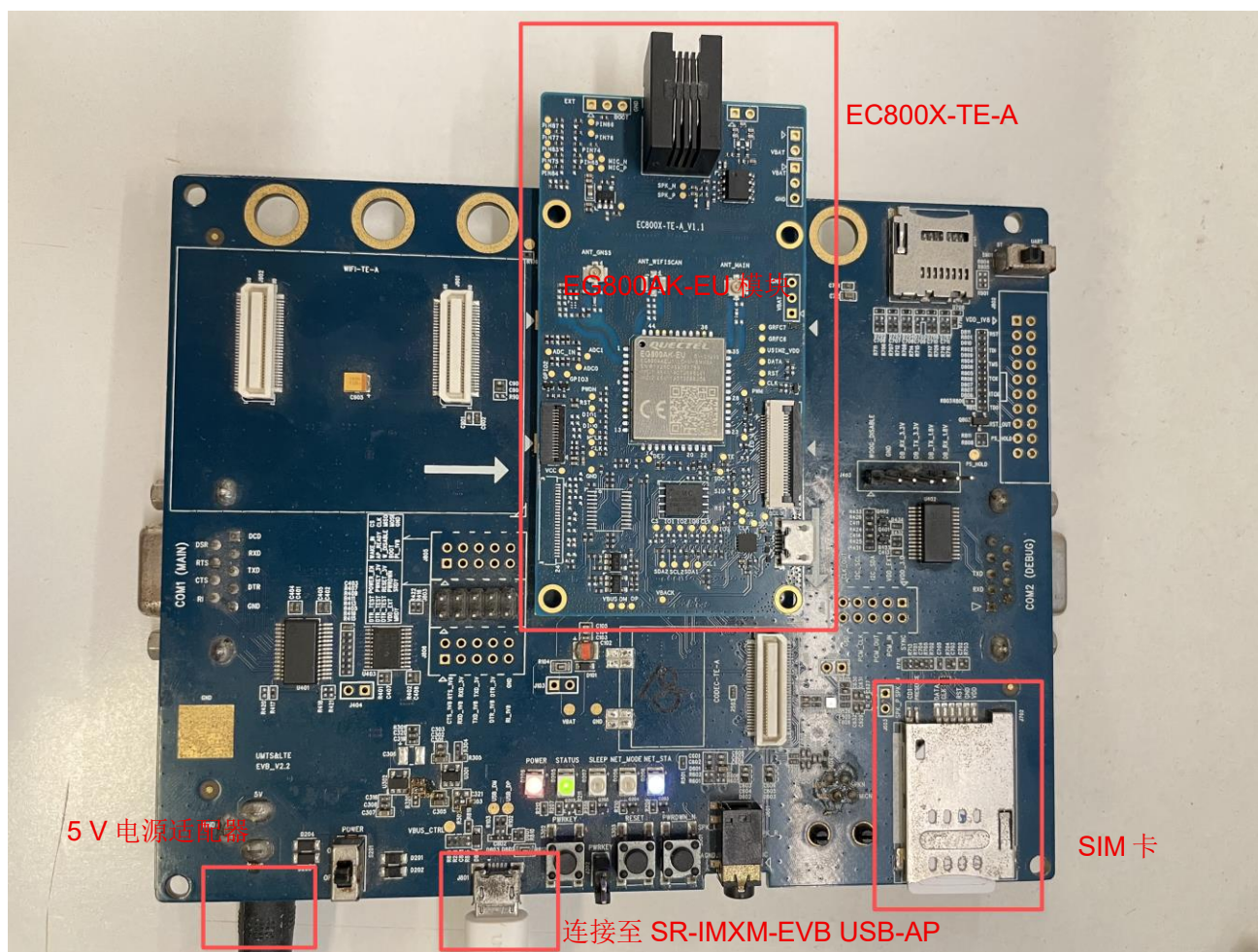


图 3: 安装 EG800AK-EU 模块和 SIM 卡到 UMTS<E-EVB

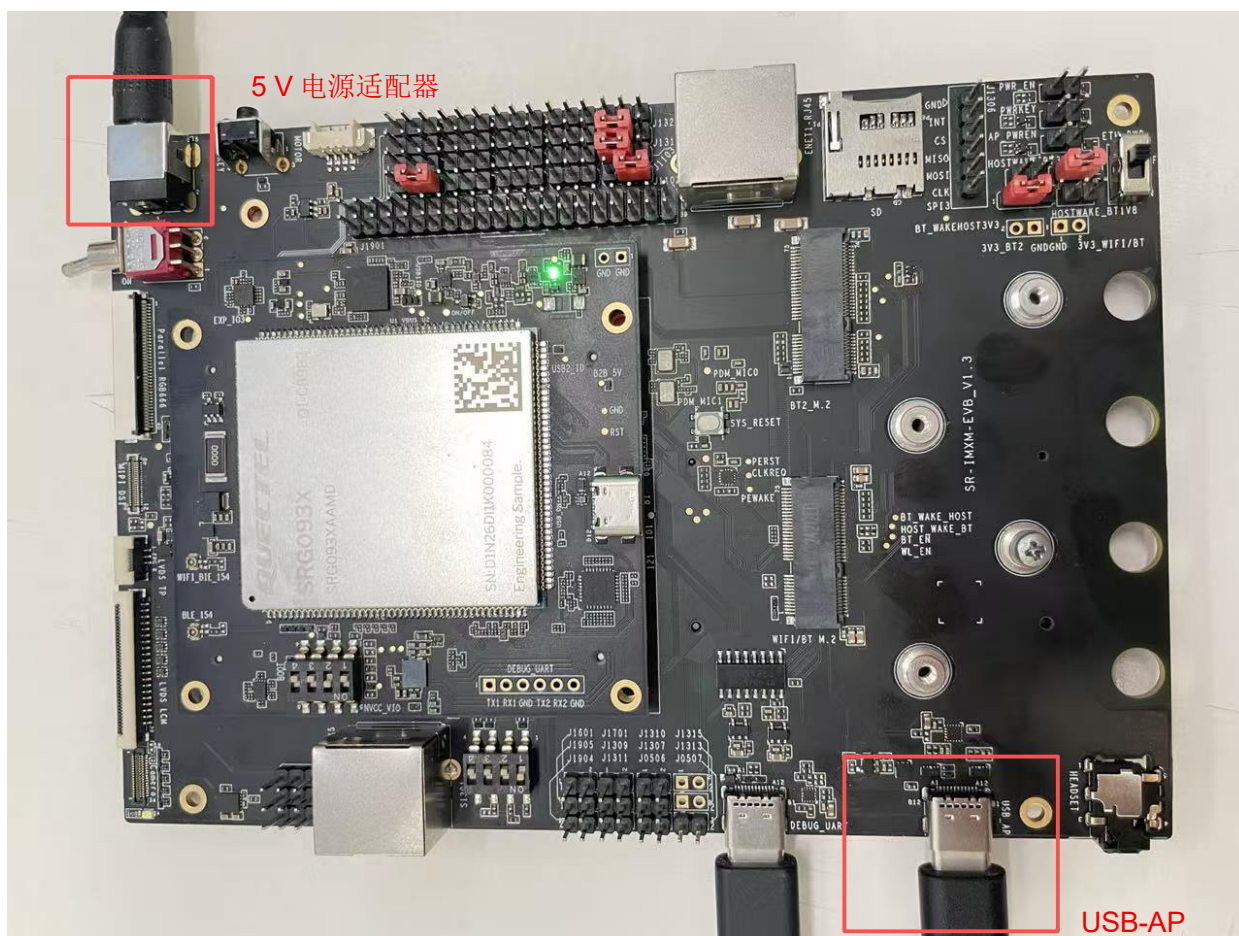


图 4: SR-IMXM-EVB 电源接口和 USB-AP 接口

2.2. 硬件改装说明

SRG091X 和 SRG093X 系列模块提供两组 USB 接口，分别为 USB1 和 USB2。在搭配 SR-IMXM-EVB 使用时，USB1 连接至 SR-IMXM-EVB 上的 USB_AP 接口，USB2 为 SR-IMXM-EVB 上的 Wi-Fi/BT M.2、BT2 M.2 和 Mini PCIe 接口的共享 USB 通路，默认连接至 Wi-Fi/BT M.2 接口。因此，如需调试安装在 Mini PCIe 接口的蜂窝模块，需按以下方式对 SR-IMXM-EVB 进行硬件改装，去掉 R1403 和 R1404 这两个电阻并焊上 R1402、R1430、R1405 和 R1409 这四个 0 欧姆电阻。



图 5：SR-IMXM-EVB 硬件改装说明

2.3. 软件环境

表 2：软件环境

类型	说明
SDK 包	SRG091X 和 SRG093X 系列模块 SDK 包
编译环境	Ubuntu 22.04 及以上版本

备注

有关获取模块 SDK 包和搭建编译环境的详细信息，请参考文档[1]。

3 移植蜂窝模块驱动

蜂窝模块驱动的移植分为 USB 转串口驱动和 USB 网络驱动两部分：

- **USB 转串口驱动：**将蜂窝模块的 USB 物理接口映射为串口设备（COM/ttyUSB），为 AT 命令交互提供基础通信信道。
- **USB 网络驱动：**在 Linux 系统中实现蜂窝模块的网络数据接入。

已参考[文档 \[2\]](#)将上述蜂窝模块驱动集成至 SRG091X 和 SRG093X 系列模块的 SDK 中，对应的补丁文件为 *Add-Cellular-Modules-Support.patch*。补丁在 SRG091X 和 SRG093X 系列模块 SDK 包中的存放路径如下：

- SRG091X 系列模块：

```
yocto/sources/meta-imx-quectel-srg091x/meta-imx-bsp/recipes-kernel/linux/linux-imx
```

- SRG093X 系列模块：

```
yocto/sources/meta-imx-quectel-srg093x/meta-imx-bsp/recipes-kernel/linux/linux-imx
```

备注

蜂窝模块驱动会持续迭代更新，SDK 内置驱动不一定为最新版本。正式产品开发阶段，请联系移远通信技术支持确认适配版本。

下文将介绍驱动的详细移植过程。

3.1. 移植 USB 转串口驱动

根据[文档 \[2\]](#)，移植 USB 转串口驱动主要有以下几个步骤：

步骤一： 开启以下内核选项。

- **CONFIG_USB_SERIAL：**USB 转串口子系统的基础框架，是后两项配置的依赖项；
- **CONFIG_USB_SERIAL_WWAN：**Modem 串口公共底层驱动，封装了 USB 数据收发、复位和休眠等逻辑，供 option 驱动调用；

- **CONFIG_USB_SERIAL_OPTION:** Modem 设备匹配驱动。该驱动通过 VID 和 PID 识别模块，并生成 `ttyUSBx` 设备节点，用以提供 AT、PPP 和 GPS 等功能端口。

在内核配置文件`[KERNEL]/arch/arm64/configs/imx_v8_defconfig`中新增或修改编译选项如下：

```
CONFIG_USB_SERIAL=y
CONFIG_USB_SERIAL_WWAN=y
CONFIG_USB_SERIAL_OPTION=y
```

步骤二： 添加蜂窝模块的 VID 和 PID。

`option` 驱动根据`[KERNEL]/drivers/usb/serial/option.c`中 `option_id[]`数组定义的 VID 和 PID 来匹配设备。因此，需要将适配蜂窝模块的 USB VID 和 PID 添加到该数组中。`option.c`中已包含 EG21-GL、EG25-GL 和 EG800AK-EU 模块的 VID 和 PID，因此只需加入 EG916Q-GL 模块的信息：

```
#define QUECTEL_VENDOR_ID          0x2c7c
/* These Quectel products use Quectel's vendor ID */
#define QUECTEL_PRODUCT_EG916Q      0x6007

static const struct usb_device_id option_ids[] = {
    .....
    { USB_DEVICE_AND_INTERFACE_INFO(QUECTEL_VENDOR_ID,
    QUECTEL_PRODUCT_EG916Q, 0xff, 0, 0) },
    .....
}
```

步骤三： ~~步骤二~~中的配置使模块的所有 USB 接口均绑定 USB 转串口 `option` 驱动程序，导致 USBNet 驱动程序接口无法工作。可以在`[KERNEL]/drivers/usb/serial/option.c`中添加以下蓝色字体内容，防止 USBNet 驱动程序接口绑定 USB 转串口 `option` 驱动程序。

```
static int option_probe(struct usb_serial *serial, const struct usb_device_id *id) {
    struct usb_wwan_intf_private *data;
    .....
    #if 1 //Added by Quectel
    if (serial->dev->descriptor.idVendor == cpu_to_le16(0x2C7C)) {
        __u16 idProduct = le16_to_cpu(serial->dev->descriptor.idProduct);
        struct usb_interface_descriptor *intf = &serial->interface->cur_altsetting->desc;
        if (intf->bInterfaceClass != 0xFF || intf->bInterfaceSubClass == 0x42) {
            //ECM, RNDIS, NCM, MBIM, ACM, UAC, ADB
            return -ENODEV;
        }
        if ((idProduct & 0xF000) == 0x0000) {
            //MDM interface 4 is QMI
            if (intf->bInterfaceNumber == 4 && intf->bNumEndpoints == 3
                && intf->bInterfaceSubClass == 0xFF && intf->bInterfaceProtocol == 0xFF)
```



```

        return -ENODEV;
    }
}
#endif
/* Store device id so we can use it during attach. */
usb_set_serial_data(serial, (void *)id);
return 0;
}

```

步骤四： 在`[KERNEL]/drivers/usb/serial/usb_wwan.c`中添加 bulk-out 传输过程中的零包处理机制，具体如下蓝色字体所示。

```

static struct urb *usb_wwan_setup_urb(struct usb_serial *serial, int endpoint,
int dir, void *ctx, char *buf, int len, void (*callback) (struct urb *))
{
    .....
    usb_fill_bulk_urb(urb, serial->dev,
usb_sndbulkpipe(serial->dev, endpoint) | dir,
buf, len, callback, ctx);
    #if 1 //Added by Quectel for zero packet
    if (dir == USB_DIR_OUT) {
        struct usb_device_descriptor *desc = &serial->dev->descriptor;
        if (desc->idVendor == cpu_to_le16(0x2C7C))
            urb->transfer_flags |= URB_ZERO_PACKET;
    }
    #endif
    return urb;
}

```

步骤五： 在`[KERNEL]/drivers/usb/serial/option.c`中添加 Reset-resume 机制，具体如下蓝色字体所示。

```

static struct usb_serial_driver option_1port_device = {
    .....
#ifdef CONFIG_PM
    .suspend = usb_wwan_suspend,
    .resume = usb_wwan_resume,
    #if 1 //Added by Quectel
    .reset_resume = usb_wwan_resume,
    #endif
#endif
};

```

备注

本章节及下文路径中的[*KERNEL*]指 SDK 中的内核代码路径。

3.2. 移植 USB 网络驱动

不同模块支持的 USB 网络驱动不同，根据文档 [2]，EG21-GL 和 EG25-GL 模块支持 GobiNet、QMI_WWAN、MBIM 等驱动，EG916Q-GL 和 EG800AK-EU 模块支持 ECM 和 RNDIS 驱动。EG21-GL 和 EG25-GL 模块默认使用 QMI_WWAN 驱动，EG916Q-GL 和 EG800AK-EU 模块默认使用 ECM 驱动。

3.2.1. 集成 QMI_WWAN 驱动

QMI_WWAN 驱动用于为移远通信 4G/5G 模块提供基于 QMI 协议的拨号和网络数据接口。本文档使用定制的 *qmi_wwan_q.c* 驱动文件，以适配蜂窝模块的 VID 和 PID。将该驱动编译为可加载内核模块后，会生成 *qmi_wwan_q.ko* 文件。

步骤一： 在内核配置文件[*KERNEL*]/arch/arm64/configs/imx_v8_defconfig 中新增或修改编译选项如下：

```
CONFIG_USB_NET_DRIVERS=y
CONFIG_USB_USBNET=y
CONFIG_USB_NET_QMI_WWAN=m
CONFIG_USB_WDM=y
```

步骤二： 按照如下内容修改[*KERNEL*]/drivers/net/usb/Makefile，请确保 *qmi_wwan_q.o* 编译条目放在原生 *qmi_wwan.o* 之前，以优先匹配定制 VID/PID。

```
# must insert qmi_wwan_q.o before qmi_wwan.o
obj-$(CONFIG_USB_NET_QMI_WWAN) += qmi_wwan_q.o
obj-$(CONFIG_USB_NET_QMI_WWAN) += qmi_wwan.o
```

步骤三： 将 *qmi_wwan_q.c* 复制到[*KERNEL*]/drivers/net/usb/目录下。

3.2.2. 集成 ECM 驱动

ECM 是一种 USB 通信设备类网络协议，用于将蜂窝模块的 USB 接口枚举为虚拟网卡。将 ECM 驱动配置为可加载内核模块后，会生成 *cdc_ether.ko* 文件。在内核配置文件[*KERNEL*]/arch/arm64/configs/imx_v8_defconfig 中新增或修改编译选项如下以集成 ECM 驱动。

```
CONFIG_USB_NET_DRIVERS=y
CONFIG_USB_USBNET=y
```

```
CONFIG_USB_NET_CDCETHER=m
```

4 移植 QConnectManager

QConnectManager 是移远通信提供的网卡拨号工具，可自动检测移远通信蜂窝模块以及其使用的拨号协议。SRG091X 系列和 SRG093X 系列模块 SDK 中已经集成了 QConnectManager，编译可产出 quectel-CM 可执行程序。对应源码在如下目录：

- SRG091X 系列模块：

```
yocto/sources/meta-imx-quectel-srg091x/meta-imx-bsp/recipes-quectel-app/quectel-cm
```

- SRG093X 系列模块：

```
yocto/sources/meta-imx-quectel-srg093x/meta-imx-bsp/recipes-quectel-app/quectel-cm
```

SRG091X 系列模块与 SRG093X 系列模块采用相同的 SDK 目录结构。本章以 SRG091X 为例进行说明。

```
yocto/sources/meta-imx-quectel-srg091x/meta-imx-bsp/recipes-quectel-app$ tree quectel-cm -L 3
quectel-cm
├── files
│   └── Quectel_QConnectManager_Linux_V1.6.8
│       ├── atc.c
│       ├── atchannel.c
│       ├── atchannel.h
│       ├── at_tok.c
│       ├── at_tok.h
│       ├── CMakeLists.txt
│       ├── configure.ac
│       ├── default.script
│       ├── default.script_ip
│       ├── device.c
│       ├── ethtool-copy.h
│       ├── GobiNetCM.c
│       ├── log
│       ├── main.c
│       ├── Makefile
│       ├── Makefile.am
│       ├── mbim-cm.c
│       └── NOTICE
```

```

|   |—— QCQCTL.h
|   |—— QCQMI.h
|   |—— QCQMUX.c
|   |—— QCQMUX.h
|   |—— qendian.h
|   |—— qlist.h
|   |—— qmap_bridge_mode.c
|   |—— QMIThread.c
|   |—— QMIThread.h
|   |—— QmiWwanCM.c
|   |—— qrtr.c
|   |—— qrtr.h
|   |—— quectel-atc-proxy.c
|   |—— quectel-mbim-proxy.c
|   |—— quectel-qmi-proxy.c
|   |—— quectel-qrtr-proxy.c
|   |—— ReleaseNote.txt
|   |—— rmnetctl.c
|   |—— udhcpc.c
|   |—— udhcpc_netlink.c
|   |—— udhcpc_script.c
|   |—— util.c
|   |—— util.h
|   |—— quectel-cm.bb

```

备注

QConnectManager 会持续迭代更新，SDK 内置集成版本并非为最新版本。进行实际产品开发时，请联系移远通信技术支持确认适配的软件版本。有关 QConnectManager 的使用说明，请参考文档[4]。

5 功能验证

5.1. 验证上网功能

完成驱动移植后，请参考[文档\[1\]](#)进行镜像编译与烧录，SR-IMXM-EVB 重新上电后，按以下步骤执行上网功能验证。

步骤一： 确认 USB 设备枚举。执行 **lsusb** 命令查看 USB 设备枚举情况，不同模块正常识别输出参考如下：

- 接入 EG21-GL 模块，示例输出如下：

```
root@quectel-srg091x:~# lsusb
Bus 001 Device 001: ID 1d6b:0002 Linux Foundation 2.0 root hub
Bus 001 Device 002: ID 2c7c:0121 Quectel Wireless Solutions Co., Ltd. EC21 LTE modem
```

- 接入 EG25-GL 模块，示例输出如下：

```
root@quectel-srg091x:~# lsusb
Bus 001 Device 001: ID 1d6b:0002 Linux Foundation 2.0 root hub
Bus 001 Device 002: ID 2c7c:0125 Quectel Wireless Solutions Co., Ltd. EC25 LTE modem
```

- 接入 EG916Q-GL 模块，示例输出如下：

```
root@quectel-srg091x:~# lsusb
Bus 001 Device 001: ID 1d6b:0002 Linux Foundation 2.0 root hub
Bus 001 Device 002: ID 2c7c:6007 Quectel Wireless Solutions Co., Ltd. EG916Q-GL
```

- 接入 EG800AK-EU 模块，示例输出如下：

```
root@quectel-srg093x:~# lsusb
Bus 001 Device 001: ID 1d6b:0002 Linux Foundation 2.0 root hub
Bus 002 Device 001: ID 1d6b:0002 Linux Foundation 2.0 root hub
Bus 002 Device 006: ID 2c7c:6002 Quectel Wireless Solutions Co., Ltd. EG800AK-EU
```

如果 **lsusb** 命令的输出中未显示蜂窝模块的 PID 和 VID，请参照[第2章](#)排查硬件、USB 供电和接线是否正常。

步骤二： 检查驱动加载和网络设备注册状态。SR-IMXM-EVB 上电后，系统会自动加载蜂窝模块的驱动并完成网络设备注册。可通过 `dmesg | grep -E 'qmi_wwan|cdc'` 和 `ifconfig` 命令确认所加载的驱动和注册的网络设备。

- 接入 EG21-GL 模块时，示例输出如下：

```
root@quectel-srg091x:~# dmesg | grep -E 'qmi_wwan|cdc'
[ 0.987925] usbcore: registered new interface driver cdc_wdm
[ 16.637947] qmi_wwan q 1-1:1.4: cdc-wdm0: USB WDM device
[ 16.641429] qmi_wwan q 1-1:1.4: Quectel EG21-G work on RawIP mode
[ 16.653323] qmi_wwan_q 1-1:1.4: rx urb size = 1520
[ 16.669976] qmi_wwan_q 1-1:1.4: wwan0: register 'qmi_wwan_q' at usb-ci_hdc.1-1, WWAN/QMI Raw IP device, 96:13:81:38:f9:a5
[ 16.682076] usbcore: registered new interface driver qmi_wwan_q
root@quectel-srg091x:~# ifconfig -a wwan0
wwan0: flags=128<NOARP> mtu 1500
    ether 96:13:81:38:f9:a5 txqueuelen 1000 (Ethernet)
    RX packets 0 bytes 0 (0.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 0 bytes 0 (0.0 B)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
```

- 接入 EG25-GL 模块时，示例输出如下：

```
root@quectel-srg091x:~# dmesg | grep -E 'qmi_wwan|cdc'
[ 0.987907] usbcore: registered new interface driver cdc_wdm
[ 16.211937] qmi_wwan q 1-1:1.4: cdc-wdm0: USB WDM device
[ 16.216217] qmi_wwan q 1-1:1.4: Quectel EG25-GL work on RawIP mode
[ 16.223583] qmi_wwan_q 1-1:1.4: rx urb size = 1520
[ 16.238106] qmi_wwan_q 1-1:1.4: wwan0: register 'qmi_wwan_q' at usb-ci_hdc.1-1, WWAN/QMI Raw IP device, a2:85:98:b1:6f:3f
[ 16.238350] usbcore: registered new interface driver qmi_wwan_q
root@quectel-srg091x:~#
root@quectel-srg091x:~#
root@quectel-srg091x:~# ifconfig -a wwan0
wwan0: flags=128<NOARP> mtu 1500
    ether a2:85:98:b1:6f:3f txqueuelen 1000 (Ethernet)
    RX packets 0 bytes 0 (0.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 0 bytes 0 (0.0 B)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
```

- 接入 EG916Q-GL 模块时，示例输出如下：

```
root@quectel-srg091x:~# dmesg | grep -E 'qmi_wwan|cdc'
[ 0.987985] usbcore: registered new interface driver cdc_wdm
[ 16.266304] cdc_ether 1-1:1.4 usb1: register 'cdc_ether' at usb-ci_hdc.1-1 CDC Ethernet Device, 22:89:84:6a:96:ab
[ 16.266577] usbcore: registered new interface driver cdc_ether
root@quectel-srg091x:~# ifconfig -a usb1
usb1: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet6 fe80::2089:84ff:fe6a:96ab prefixlen 64 scopeid 0x20<link>
    ether 22:89:84:6a:96:ab txqueuelen 1000 (Ethernet)
    RX packets 0 bytes 0 (0.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 22 bytes 3871 (3.7 KiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
```

- 接入 EG800AK-EU 模块时，示例输出如下：

```
root@quectel-srg093x:~# dmesg | grep -E 'qmi_wwan|cdc'
[ 1.015604] usbcore: registered new interface driver cdc_wdm
[ 12.392987] cdc_ether 2-1:1.0 usb0: register 'cdc_ether' at usb-ci hsrc.0-1, CDC Ethernet Device, ae:0c:29:a3:9b:6d
[ 12.393185] usbcore: registered new interface driver cdc_ether
root@quectel-srg093x:~# ifconfig -a usb0
usb0: flags=28605<UP,BROADCAST,RUNNING,MULTICAST,DYNAMIC> mtu 1500
    inet 169.254.117.112 netmask 255.255.0.0 broadcast 169.254.255.255
    inet6 fe80::ac0c:29ff:fea3:9b6d prefixlen 64 scopeid 0x20<link>
    ether ae:0c:29:a3:9b:6d txqueuelen 1000 (Ethernet)
    RX packets 5 bytes 316 (316.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 56 bytes 12784 (12.4 KiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
```

步骤四： 使用 quectel-CM 拨号。该工具可自动识别接入的蜂窝模块并匹配对应驱动，完成拨号和建立数据链路。

- 接入 EG21-GL 模块，拨号过程日志输出如下：

```
root@quectel-srg091x:~# quectel-CM &
[1] 528
root@quectel-srg091x:~# [03-01_08:43:07:778] QConnectManager_Linux_V1.6.8
[03-01_08:43:07:778] Find /sys/bus/usb/devices/1-1 idVendor=0x2c7c idProduct=0x121,
bus=0x001, dev=0x002
[03-01_08:43:07:779] Auto find qmichannel = /dev/cdc-wdm0
[03-01_08:43:07:779] Auto find usbnet_adapter = wwan0
[03-01_08:43:07:779] netcard driver = qmi_wwan_q, driver version = V1.4.0
[03-01_08:43:07:780] Modem works in QMI mode
[03-01_08:43:07:794] cdc_wdm_fd = 7
[03-01_08:43:07:879] Get clientWDS = 5
[03-01_08:43:07:911] Get clientDMS = 1
[03-01_08:43:07:943] Get clientNAS = 2
[03-01_08:43:07:975] Get clientUIM = 2
[03-01_08:43:08:007] Get clientWDA = 1
[03-01_08:43:08:039] requestBaseBandVersion EG21GLGAR07A02M1G
[03-01_08:43:08:167] requestGetSIMStatus SIMStatus: SIM_READY
[03-01_08:43:08:231] requestGetProfile[pdp:1 index:1] 3gnet///0/IPV4V6
[03-01_08:43:08:263] requestRegistrationState2 MCC: 460, MNC: 1, PS: Attached, DataCap: LTE
[03-01_08:43:08:295] requestQueryDataCall IPv4ConnectionStatus: DISCONNECTED
[03-01_08:43:08:295] ip addr flush dev wwan0
[03-01_08:43:08:305] ip link set dev wwan0 down
[03-01_08:43:08:359] requestSetupDataCall WdsConnectionIPv4Handle: 0x86db93b0
[03-01_08:43:08:487] ip link set dev wwan0 up
[03-01_08:43:08:505] busybox udhcpc -f -n -q -t 5 -i wwan0
udhcpc: started, v1.36.1
Dropped protocol specifier '.udhcpc' from 'wwan0.udhcpc'. Using 'wwan0' (ifindex=6).
udhcpc: broadcasting discover
```

```
udhcpc: broadcasting select for 10.60.201.233, server 10.60.201.234
udhcpc: lease of 10.60.201.233 obtained from 10.60.201.234, lease time 7200
[03-01_08:43:08:725] /etc/udhcpc.d/50default: Adding DNS 218.104.111.122
[03-01_08:43:08:725] /etc/udhcpc.d/50default: Adding DNS 218.106.127.114
Dropped protocol specifier '.udhcpc' from 'wwan0.udhcpc'. Using 'wwan0' (ifindex=6).
```

- 接入 EG25-GL 模块，拨号过程日志输出如下：

```
root@quectel-srg091x:~# quectel-CM &
[1] 558
root@quectel-srg091x:~# [02-27_18:26:39:559] QConnectManager_Linux_V1.6.8
[02-27_18:26:39:560] Find /sys/bus/usb/devices/1-1 idVendor=0x2c7c idProduct=0x125,
bus=0x001, dev=0x002
[02-27_18:26:39:560] Auto find qmichannel = /dev/cdc-wdm0
[02-27_18:26:39:561] Auto find usbnet_adapter = wwan0
[02-27_18:26:39:561] netcard driver = qmi_wwan, driver version = 6.6.52-lts-next-g163b09e0529a
[02-27_18:26:39:561] Modem works in QMI mode
[02-27_18:26:39:570] cdc_wdm_fd = 7
[02-27_18:26:39:654] Get clientWDS = 5
[02-27_18:26:39:686] Get clientDMS = 1
[02-27_18:26:39:718] Get clientNAS = 2
[02-27_18:26:39:750] Get clientUIM = 2
[02-27_18:26:39:782] Get clientWDA = 1
[02-27_18:26:39:814] requestBaseBandVersion EG25GLGAR07A03M1G
[02-27_18:26:39:942] requestGetSIMStatus SIMStatus: SIM_READY
[02-27_18:26:40:006] requestGetProfile[pdp:1 index:1] 3gnet///0/IPV4V6
[02-27_18:26:40:039] requestRegistrationState2 MCC: 460, MNC: 1, PS: Attached, DataCap: LTE
[02-27_18:26:40:071] requestQueryDataCall IPv4ConnectionStatus: DISCONNECTED
[02-27_18:26:40:071] ip addr flush dev wwan0
[02-27_18:26:40:081] ip link set dev wwan0 down
[02-27_18:26:40:135] requestSetupDataCall WdsConnectionIPv4Handle: 0x86d5c330
[02-27_18:26:40:263] ip link set dev wwan0 up
[02-27_18:26:40:276] busybox udhcpc -f -n -q -t 5 -i wwan0
udhcpc: started, v1.36.1
Dropped protocol specifier '.udhcpc' from 'wwan0.udhcpc'. Using 'wwan0' (ifindex=6).
udhcpc: broadcasting discover
udhcpc: broadcasting select for 10.60.201.155, server 10.60.201.156
udhcpc: lease of 10.60.201.155 obtained from 10.60.201.156, lease time 7200
[02-27_18:26:40:493] /etc/udhcpc.d/50default: Adding DNS 218.104.111.122
[02-27_18:26:40:493] /etc/udhcpc.d/50default: Adding DNS 218.106.127.114
Dropped protocol specifier '.udhcpc' from 'wwan0.udhcpc'. Using 'wwan0' (ifindex=6).
```

- 接入 EG916Q-GL 模块，拨号过程日志输出如下：

```

root@quectel-srg091x:~# quectel-CM &
[1] 504
root@quectel-srg091x:~# [02-27_18:21:22:116] QConnectManager_Linux_V1.6.8
[02-27_18:21:22:117] Find /sys/bus/usb/devices/1-1 idVendor=0x2c7c idProduct=0x6007,
bus=0x001, dev=0x002
[02-27_18:21:22:117] Auto find qmichannel = /dev/ttyUSB3
[02-27_18:21:22:117] Auto find usbnet_adapter = usb1
[02-27_18:21:22:117] netcard driver = cdc_ether, driver version = 6.6.52-lts-next-g163b09e0529a
[02-27_18:21:22:117] Modem works in ECM_RNDIS_NCM mode
.....
[02-27_18:21:23:138] AT> AT+QCFG="usbnet"
[02-27_18:21:23:141] AT< +QCFG: "usbnet",1
[02-27_18:21:23:141] AT< OK
.....
[02-27_18:21:23:221] ip addr flush dev usb1
[02-27_18:21:23:232] ip link set dev usb1 down
[02-27_18:21:23:259] AT> AT+CGACT?
[02-27_18:21:23:263] AT< +CGACT: 1,1
[02-27_18:21:23:263] AT< OK
[02-27_18:21:23:263] AT> AT+QNETDEVCTL=1,1,1
[02-27_18:21:23:269] AT< OK
[02-27_18:21:23:269] AT> AT+CGPADDR=1
[02-27_18:21:23:354] AT< +CGPADDR: 1,"10.10.36.212,36.8.132.79.13.70.96.21.0.0.0.0.0.0.1"
[02-27_18:21:23:354] AT< OK
[02-27_18:21:23:354] requestGetIPAddress 10.10.36.212
[02-27_18:21:23:354] AT> at+cops?
[02-27_18:21:23:357] AT< +COPS: 0,2,"46001",7
[02-27_18:21:23:357] AT< OK
[02-27_18:21:23:357] AT> at+qeng="servingcell"
[02-27_18:21:23:375] AT< +QENG:
"servingcell","NOCONN","LTE","FDD",460,01,1A3171A,416,1600,3,5,5,F415,-72,-9,-43,15,56
[02-27_18:21:23:376] AT< OK
.....
[02-27_18:21:23:383] ip link set dev usb1 up
[02-27_18:21:23:403] busybox udhcpc -f -n -q -t 5 -i usb1
udhcpc: started, v1.36.1
Dropped protocol specifier '.udhcpc' from 'usb1.udhcpc'. Using 'usb1' (ifindex=6).
udhcpc: broadcasting discover
udhcpc: broadcasting select for 10.10.36.212, server 10.10.36.213
udhcpc: lease of 10.10.36.212 obtained from 10.10.36.213, lease time 86400
Error: ipv4: Address already assigned.
[02-27_18:21:24:441] /etc/udhcpc.d/50default: Adding DNS 218.104.111.122

```

```
[02-27_18:21:24:441] /etc/udhcpc.d/50default: Adding DNS 218.106.127.114
Dropped protocol specifier '.udhcpc' from 'usb1.udhcpc'. Using 'usb1' (ifindex=6).
```

- 接入 EG800AK-EU 模块，拨号过程日志输出如下：

```
root@quectel-srg093x:~# quectel-CM &
[1] 548
[02-27_18:41:49:275] QConnectManager_Linux_V1.6.8
root@quectel-srg093x:~# [02-27_18:41:49:280] Find /sys/bus/usb/devices/2-1 idVendor=0x2c7c
idProduct=0x6002, bus=0x002, dev=0x002
[02-27_18:41:49:281] Auto find qmichannel = /dev/ttyUSB1
[02-27_18:41:49:281] Auto find usbnet_adapter = usb0
[02-27_18:41:49:281] netcard driver = cdc_ether, driver version = 6.6.52-lts-next-g60def389bb16-
d
[02-27_18:41:49:281] Modem works in ECM_RNDIS_NCM mode
.....
[02-27_18:41:50:294] AT> AT+QCFG="usbnet"
[02-27_18:41:50:297] AT< +QCFG: "usbnet",1
[02-27_18:41:50:297] AT< OK
.....
[02-27_18:41:50:329] ip addr flush dev usb0
[02-27_18:41:50:338] ip link set dev usb0 down
[02-27_18:41:50:356] AT> AT+CGACT?
[02-27_18:41:50:359] AT< +CGACT: 1,1
[02-27_18:41:50:359] AT< OK
[02-27_18:41:50:360] AT> AT+QNETDEVCTL=1,1,1
[02-27_18:41:52:358] AT< OK
[02-27_18:41:52:359] AT> AT+CGPADDR=1
[02-27_18:41:52:359] AT< +QNETDEVSTATUS: 1
[02-27_18:41:52:362] AT< +CGPADDR: 1,"10.158.17.165","2409:8931:4ED:4C0A::1"
[02-27_18:41:52:362] AT< OK
[02-27_18:41:52:362] requestGetIPAddress 10.158.17.165
[02-27_18:41:52:362] AT> at+cops?
[02-27_18:41:52:364] AT< +COPS: 0,2,"46000",7
[02-27_18:41:52:364] AT< OK
[02-27_18:41:52:364] AT> at+qeng="servingcell"
[02-27_18:41:52:369] AT< +QENG:
"servingcell","NOCONN","LTE","FDD",460,00,848459E,207,3590,8,3,3,550B,-110,-3,-90,16,14
[02-27_18:41:52:369] AT< OK
.....
[02-27_18:41:52:374] ip link set dev usb0 up
[02-27_18:41:52:387] busybox udhcpc -f -n -q -t 5 -i usb0
udhcpc: started, v1.36.1
Dropped protocol specifier '.udhcpc' from 'usb0.udhcpc'. Using 'usb0' (ifindex=4).
```

```
udhcpc: broadcasting discover
udhcpc: broadcasting select for 10.158.17.165, server 169.254.0.1
udhcpc: lease of 10.158.17.165 obtained from 169.254.0.1, lease time 86400
Error: ipv4: Address already assigned.
[02-27_18:41:52:794] /etc/udhcpc.d/50default: Adding DNS 211.138.180.4
[02-27_18:41:52:794] /etc/udhcpc.d/50default: Adding DNS 211.138.180.5
Dropped protocol specifier '.udhcpc' from 'usb0.udhcpc'. Using 'usb0' (ifindex=4).
```

拨号成功后，可执行 **ifconfig wwan0** 和 **ip route show** 命令分别查看网络接口的 IP 地址及路由信息，预期输出如下：

```
root@quectel-srg091x:~# ifconfig wwan0
wwan0: flags=4305<UP,POINTOPOINT,RUNNING,NOARP,MULTICAST> mtu 1500
    inet 10.60.201.155 netmask 255.255.255.248 destination 10.60.201.155
    inet6 fe80::2f7b:f9d:20b6:2bcf prefixlen 64 scopeid 0x20<link>
    unspec 00-00-00-00-00-00-00-00-00-00-00-00-00-00-00-00 txqueuelen 1000 (UNSPEC)
    RX packets 16 bytes 2317 (2.2 KiB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 51 bytes 6879 (6.7 KiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

root@quectel-srg091x:~# ip route show
default via 10.60.201.156 dev wwan0 metric 10
10.60.201.152/29 dev wwan0 proto kernel scope link src 10.60.201.155
```

步骤五： 使用 AT 命令查询注网信息。

SR-IMXM-EVB 板载的 DEBUG_UART 串口转换芯片提供一路与蜂窝模块串口直连的通道，可用于下发 AT 命令并与蜂窝模块交互。使用 USB Type-C 线将 DEBUG_UART 接口连接至 PC。在 Windows 设备管理器中，可识别到 4 个虚拟 COM 端口，如下图所示。

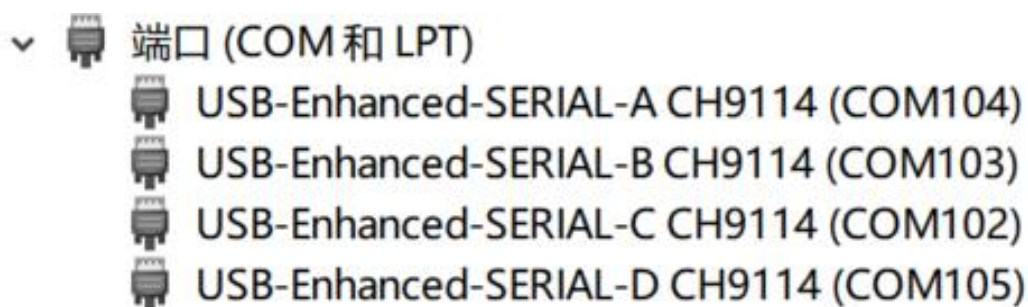


图 6：串口信息

4 个 COM 串口对应信息如下：

表 3：串口信息

串口名称	描述
USB-Enhanced-SERIAL-A	SRG091X 和 SRG093X 系列模块 A 核串口
USB-Enhanced-SERIAL-B	SRG093X 系列模块 M33 核串口
USB-Enhanced-SERIAL-C	蜂窝模块 AT 命令端口
USB-Enhanced-SERIAL-D	未使用

使用串口工具打开 USB-Enhanced-SERIAL-C 对应的 COM 端口，并将波特率设置为 115200，即可向蜂窝模块下发 AT 命令。

- 发送 **AT+CPIN?**：查询 SIM 卡状态。返回**+CPIN: READY** 表示 SIM 卡状态正常。
- 发送 **AT+CREG?**：查询蜂窝网络注册状态。返回**+CREG: 0,1** 表示模块已注册到本地网络。

```

AT+CPIN?
+CPIN: READY

OK
AT+CREG?
+CREG: 0,1

OK
    
```

5.2. 验证语音通话功能

本章节以 SRG091X-TE-A 搭配 SR-IMXM-EVB 和 EG25-GL Mini PCIe 模块为例，介绍如何进行语音通话功能验证。

5.2.1. 硬件条件说明

进行语音通话验证前，需确保 SR-IMXM-EVB 的耳机接口能正常输出音频，即 SRG091X-TE-A 上的 Codec WM8962 正常工作。根据硬件设计，SRG091X-TE-A 出厂默认配置不支持 Codec 功能，需修改板上电阻完成硬件改动。如下图所示，将红框内的四颗电阻由水平放置修改为竖直摆放：



图 7：对 SRG091X-TE-A 进行硬件改动以支持 Codec

SR-IMXM-EVB 上的 Mini PCIe 接口和 Wi-Fi/BT M.2 接口共用 SRG091X SoC 的 SAI1 接口，PCM 通路由 SR-IMXM-EVB 底板上的 S1301 拨码开关第 1 位控制。进行蜂窝语音通话验证时，需将 S1301 拨码开关的第 1 位拨至“ON”，使 SAI1 接口连接至蜂窝模块的 PCM 通路。

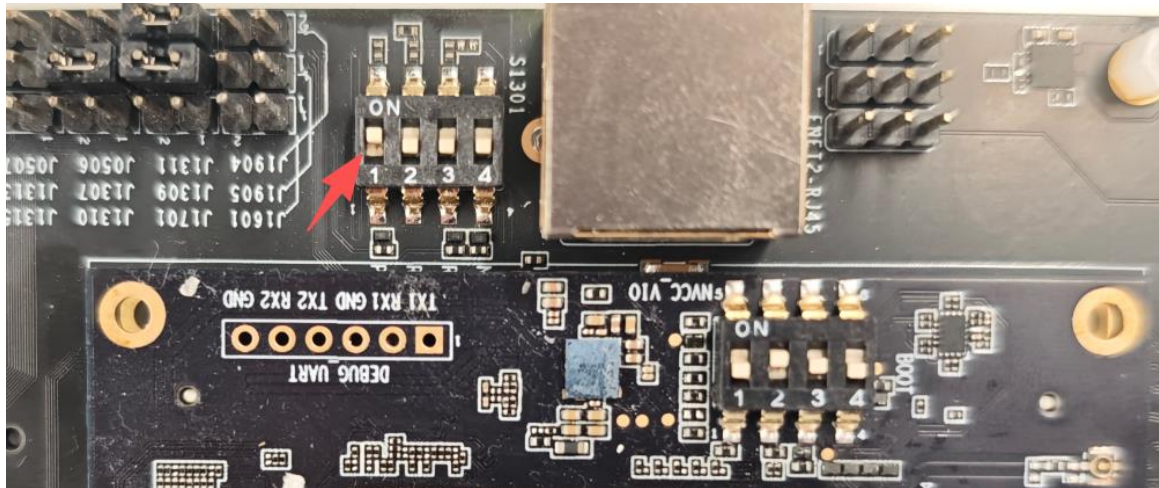


图 8：通过拨码开关切换 PCM 通路

备注

1. SRG091X-W 和 SRG093X-W 模块的 SAI1_RXD0 引脚被复用为 PMIC 中断输入引脚，不支持使用对应型号的 TE-A 搭配 SR-IMXM-EVB 进行蜂窝模块语音通话验证。
2. SRG093X-TE-A 出厂默认支持 Codec 功能。如使用安装 SRG093X-TE-A 的 SR-IMXM-EVB 进行语音通话验证，则无需进行硬件改装。

5.2.2. 使能 SAI1 接口和 sound-bt-sco 声卡

SDK 默认已使能 SAI1 接口和 sound-bt-sco 声卡，相关补丁文件为 *Enable-bt-sco-Sound-Card.patch*。不同模块的补丁存放路径如下：

- SRG091X 系列模块：

```
yocto/sources/meta-imx-quectel-srg091x/meta-imx-bsp/recipes-kernel/linux/linux-imx
```

- SRG093X 系列模块：

```
yocto/sources/meta-imx-quectel-srg093x/meta-imx-bsp/recipes-kernel/linux/linux-imx
```

以下为部分关键设备树配置：

```
arch/arm64/boot/dts/freescale/imx91-quectel-srg091x-common.dts
/ {
    bt_sco_codec: bt_sco_codec {
        #sound-dai-cells = <1>;
        compatible = "linux,bt-sco";
    };

    sound-bt-sco {
        compatible = "simple-audio-card";
        simple-audio-card,name = "bt-sco-audio";
        simple-audio-card,format = "dsp_a";
        simple-audio-card,bitclock-inversion;
        simple-audio-card,frame-master = <&btcpu>;
        simple-audio-card,bitclock-master = <&btcpu>;
        status = "okay";
        btcpu: simple-audio-card,cpu {
            sound-dai = <&sai1>;
            dai-tdm-slot-num = <2>;
            dai-tdm-slot-width = <16>;
        };

        simple-audio-card,codec {
            sound-dai = <&bt_sco_codec 1>;
        };
    };
};

&sai1 {
    #sound-dai-cells = <0>;
```

```
pinctrl-names = "default", "sleep";
pinctrl-0 = <&pinctrl_sai1>;
pinctrl-1 = <&pinctrl_sai1_sleep>;
assigned-clocks = <&clk IMX91_CLK_SAI1>;
assigned-clock-parents = <&clk IMX91_CLK_AUDIO_PLL>;
assigned-clock-rates = <12288000>;
fsl,sai-mclk-direction-output;
status = "okay";
};
```

5.2.3. 配置蜂窝模块 PCM 接口

根据[文档 \[3\]](#)，蜂窝模块的 PCM 接口支持配置主从模式、采样率等参数。本章节以 SRG091X 模块配置为主模式、EG25-GL 模块配置为从模式为示例。通过 EG25-GL 模块的 AT 命令端口下发 **AT+QDAI=1,1,0,2,0,1,1,1**，若返回 **OK** 表示命令执行成功。需注意，该配置在 EG25-GL 模块重新启动后生效。因此，下发命令后需重新启动 EG25-GL 模块（即开发板断电后重新上电）。模块重启完成并输出 **RDY** 后，执行 **AT+QDAI?** 查询当前配置。若查询结果与设定值一致，则表示配置已成功生效。具体配置如下：

```
AT+QDAI=1,1,0,2,0,1,1,1
OK

RDY
AT+QDAI?
+QDAI: 1,1,0,2,0,1,1,1

OK
```

备注

不同蜂窝模块配置 PCM 参数的 AT 命令可能不同，详情请参考[文档 \[3\]](#)。

5.2.4. 验证语音通话

步骤一： 登录 SRG091X 的 Linux 命令行终端，执行以下命令启动 PipeWire、WirePlumber 和双向音频回环。

```
root@quectel-srg091x:~# systemctl --user start pipewire wireplumber
root@quectel-srg091x:~# pw-loopback --capture alsa_input.platform-sound-bt-sco.mono-fallback -
-playback alsa_output.platform-sound-wm8962.stereo-fallback &
root@quectel-srg091x:~# pw-loopback --capture alsa_input.platform-sound-wm8962.stereo-
fallback --playback alsa_output.platform-sound-bt-sco.mono-fallback &
```

步骤二： 将耳机插入 SR-IMXM-EVB 的耳机接口。

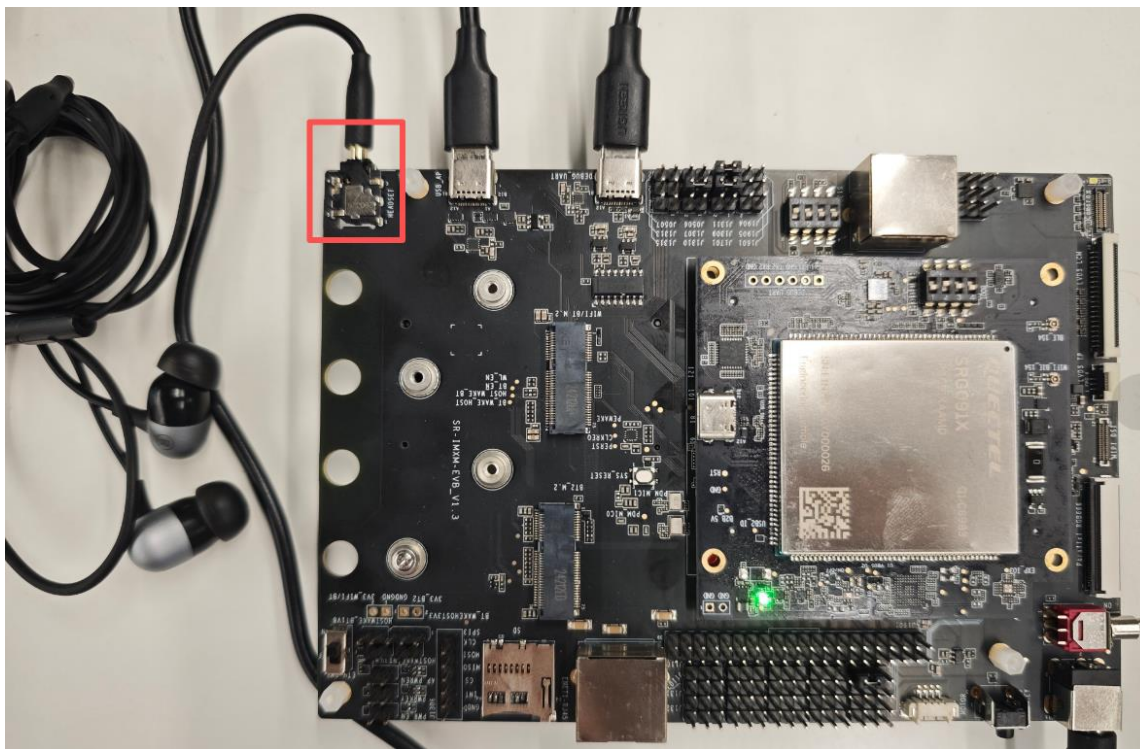


图 9：将耳机插入 SR-IMXM-EVB 的耳机接口

步骤三： 在蜂窝模块的 AT 命令端口输入 **ATDxxxxxx**;发起呼叫。实际操作时，需将 **xxxxxx** 替换为实际呼叫的手机号码。

步骤四： 使用被叫手机接听来电。双方分别讲话，确认两端语音是否均能清晰听到对方声音。若两端对话清晰，证明语音通路工作正常。

6 附录 参考文档及术语缩写

表 4: 参考文档

文档名称
[1] Quectel_SRG091X&SRG093X 系列_编译&烧录指导
[2] Quectel_UMTS_LTE_5G_Linux_USB_Driver_用户指导
[3] Quectel_EC2x&EG2x&EG9x&EM05_Series_AT_Commands_Manual
[4] Quectel_QConnectManager_Linux_用户指导

表 5: 术语缩写

缩写	英文全称	中文全称
ACM	Abstract Control Model	抽象控制模型
ADB	Android Debug Bridge	Android 调试桥
Codec	Coder-Decoder	编解码器
COM	Communication Port	通信端口
DNS	Domain Name System	域名系统
ECM	Ethernet Control Model	以太网控制模型
GPS	Global Positioning System	全球定位系统
ID	Identifier	标识符
IP	Internet Protocol	网际协议
MBIM	Mobile Broadband Interface Model	移动宽带接口模型
NCM	Network Control Model	网络控制模型
PCM	Pulse-Code Modulation	脉冲编码调制

PID	Product ID	产品标识符
PMIC	Power Management Integrated Circuit	电源管理集成电路
PPP	Point-to-Point Protocol	点对点协议
QMI	Qualcomm Messaging Interface	高通消息接口
RNDIS	Remote Network Driver Interface Specification	远程网络驱动接口规范
SAI	Serial Audio Interface	串行音频接口
SDK	Software Development Kit	软件开发工具包
SIM	Subscriber Identity Module	用户身份识别模块
SoC	System on Chip	片上系统
UAC	USB Audio Class	USB 音频设备类
UART	Universal Asynchronous Receiver/Transmitter	通用异步收发器
URB	USB Request Block	USB 请求块
USB	Universal Serial Bus	通用串行总线
VID	Vendor ID	厂商标识符
WWAN	Wireless Wide Area Network	无线广域网